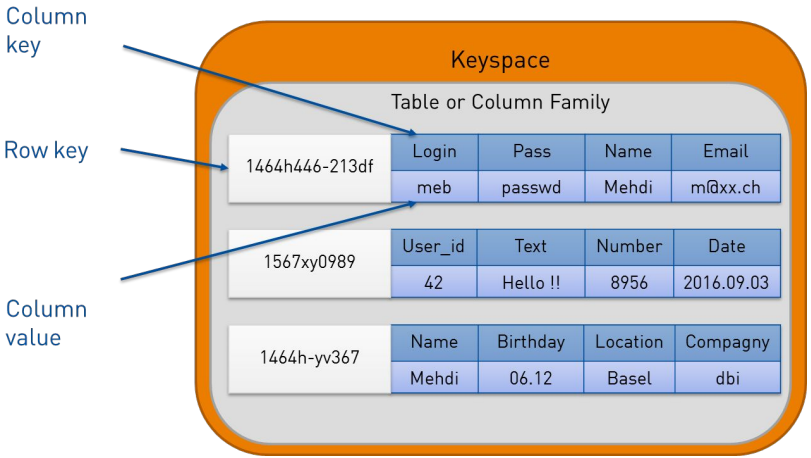
NoSQL Databases

Francesco Pelacani
2025-2026

Column-Family Stores

What is a Column-Family Data Store?

Column-family stores allow you to store data with keys mapped to values and the **values grouped into multiple column families**, each column family being a map of data.



Features

- **Consistency** → *Quorum consistency*. Commit log > memtable > SSTables
- **Transactions** → Atomicity at row level
- **Availability** → Highly available, every node is a peer in the cluster
- **Query features** → Column families should be optimized for reading, no rich QL
- **Structure of data** → Each row is sorted by column names
- **Scaling** → Add more nodes to support more writes and reads

Usage

Suitable Use Cases

1. Event Logging
2. Blogging Platforms
3. Expiring Usage
4. Counters

When Not to Use

1. When you need ACID transactions for writes and reads
2. If you need the db to aggregate data using queries (SUM/AVG)

Apache Cassandra

What is Apache Cassandra?

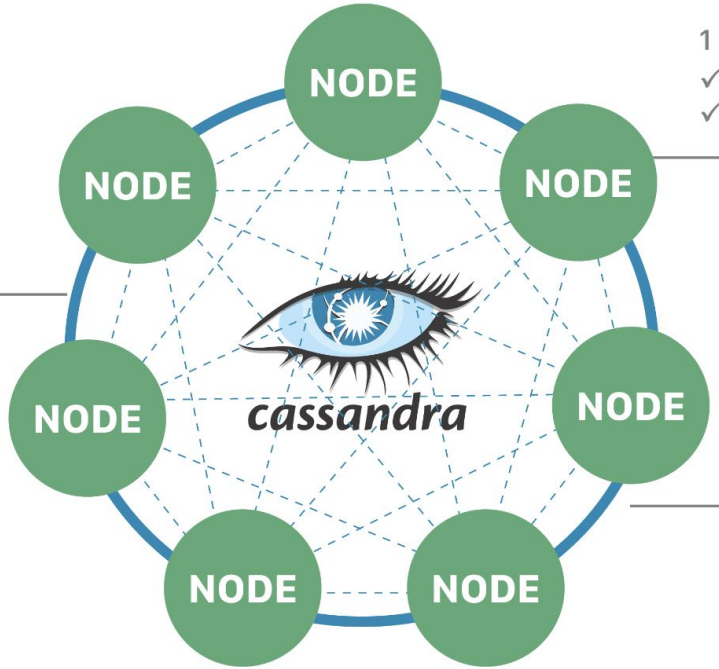
Is an open source NoSQL distributed database trusted for its **scalability** and **high availability** without compromising **performance** designed at Facebook.

It provides linear scalability and fault-tolerance on commodity hardware.

Pillars

- **Hybrid** → Masterless architecture and low latency make Cassandra withstand and entire data center outage with no data loss
- **Fault tolerant** → Failed nodes can be replaced with no downtime
- **Configurable Consistency** → each update can have sync or async replication. Highly available async operations are optimized with Hinted Handoff and Read Repair
- **Distributed** → Every node in the cluster is identical
- **Scalable** → Reads and writes both increase linearly as new machines are added, with no downtime
- **Elastic** → Streams data between nodes during scaling operations

Distribution Power and Resilience



1 Installation = 1 NODE
✓ Capacity + 1TB
✓ Throughput: 3000 Tx/sec/core

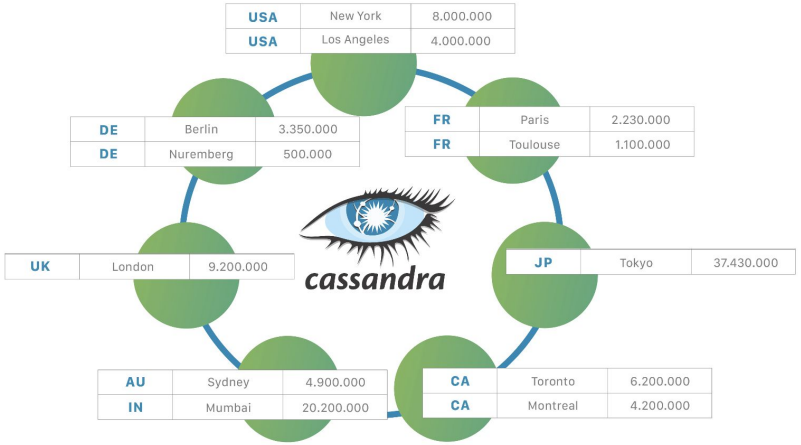
DataCenter | Ring

Communication:
✓ Gossiping

More Nodes
==
More Power

COUNTRY	CITY	POPULATION
USA	New York	8.000.000
USA	Los Angeles	4.000.000
FR	Paris	2.230.000
DE	Berlin	3.350.000
UK	London	9.200.000
AU	Sydney	4.900.000
DE	Nuremberg	500.000
CA	Toronto	6.200.000
CA	Montreal	4.200.000
FR	Toulouse	1.100.000
JP	Tokyo	37.430.000
IN	Mumbai	20.200.000

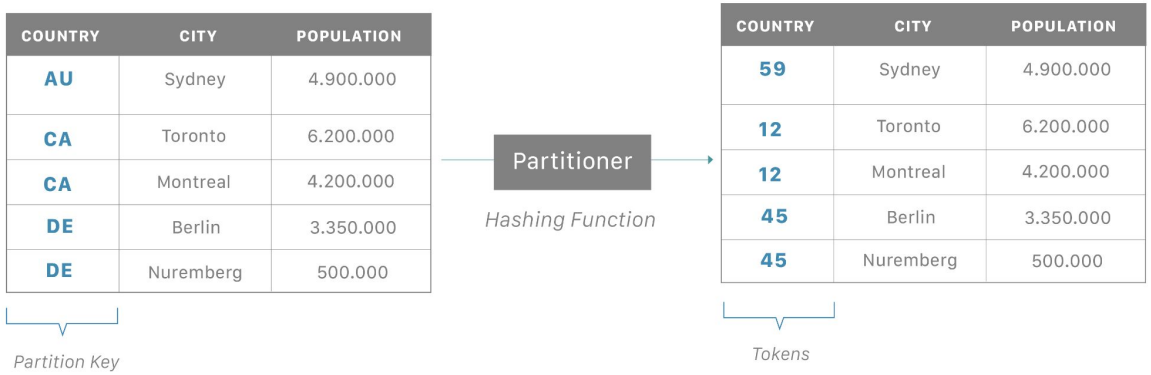
Partition Key



Partitions

Data is automatically distributed via **partitions**.
Each node owns a particular set of tokens, and Cassandra distributes data based on the ranges of these tokens across the cluster.

When data is inserted into the cluster, the *coordinator* apply a **hash function** to the partition key to determine what node (based on the token range) will get the data.

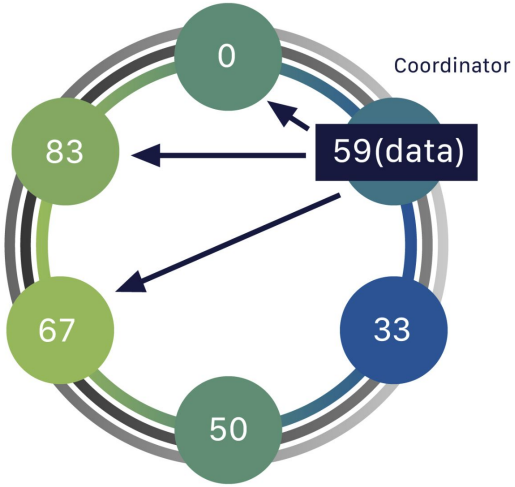


Data Replication

One piece of data can be replicated to multiple (*replica*) nodes, ensuring **reliability** and **fault tolerance**.

Replication ensures that data isn't lost.

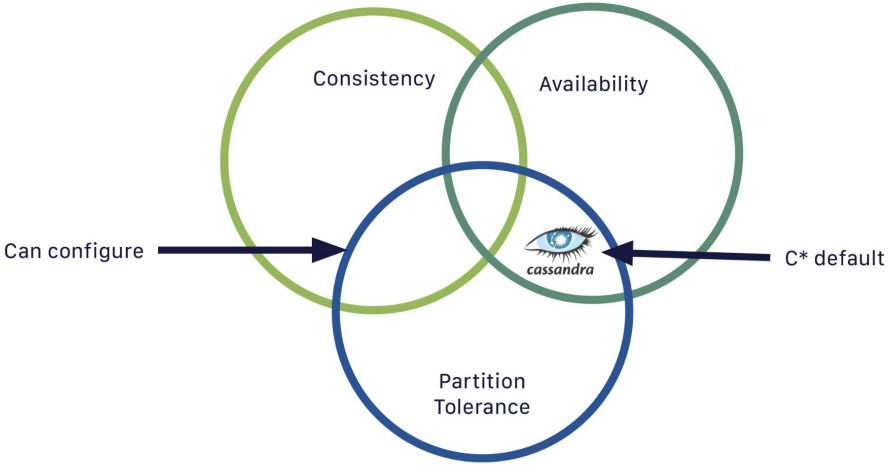
RF = 3



Tuning Consistency - 1

Cassandra is by default an **AP** (Available Partition-tolerant) database, hence it is “always on”.

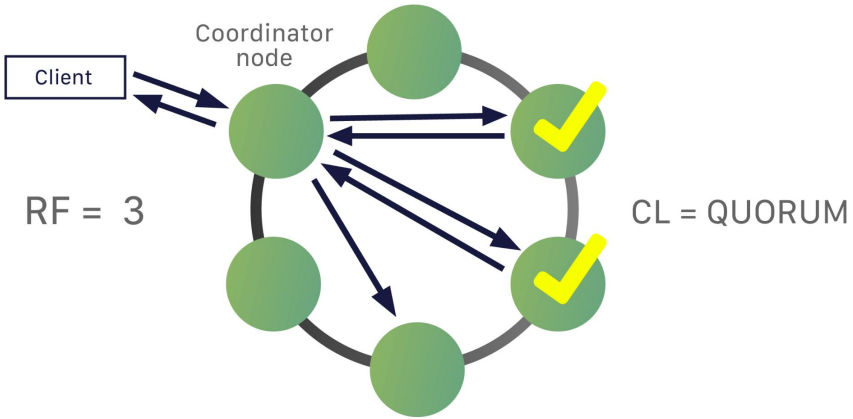
Consistency level is represented by the minimum number of nodes that must acknowledge a read or write operation to the coordinator before the operation is considered successful.



Tuning Consistency - 2

Consistency Level (CL) depends on replication factor.

We have a CL=QUORUM (Quorum referring to majority, 2 replicas in this case or RF/2 +1)



Deployment Agnostic

Cassandra is deployment agnostic.

It doesn’t care where you put it – on prem, a cloud provider, multiple cloud providers. You can use a combination of those for a single database.

Main Concepts

Keyspace

Keyspaces serve as **containers for tables**.

Specifies the replication strategy for a dataset across different datacenters.

```
CREATE KEYSPACE cycling
  WITH REPLICATION = {
   'class' : 'NetworkTopologyStrategy',
   'datacenter1' : 1
  } ;
```

Replication refers to the number of copies stored within a cluster.

Working with Keyspaces

There are 2 ways to work with objects in a keyspace.

1. Switch to a keyspace with the USE command

```
USE showroom;
DESCRIBE TABLE inventory;
```

2. Reference the object with the keyspace name and a dot (.)

```
DESCRIBE TABLE showroom.inventory;
```

Table

Tables are **composed of rows and columns**. Similar to RDBMS tables.

Columns define the typed schema for a single datum in a table.

Tables are partitioned based on the columns provided in the partition key.

Cassandra tables can flexibly add new columns to tables with zero downtime.

Row and Column

A Row **contains a collection of columns** identified by a unique primary key made up of the partition key and optionally additional clustering keys.

A Column is **a single datum with a type** which belongs to a row.

The conjunction of a Row and a Column is a **Cell**.

Create a Table

```
CREATE TABLE inventory (
     id int PRIMARY KEY,
     make text,
     model text,
     year int
);
```

Work with Tables

Insert data

```
INSERT INTO inventory(id, make, model, year) values(1001, ‘Ford’, ‘Mustang’, 1968);
INSERT INTO inventory(id, make, model, year) values(1006, ‘Ford’, ‘Mustang’, 1971);
```

Retrieve data

```
SELECT * FROM inventory;
```

Partitions

Defines the mandatory **part of the primary key** all rows in Cassandra must have **to identify the node in a cluster where the row is stored**.

Understanding partitions is crucial to effectively using Cassandra:

- Partitioning affects availability and performance
- Data modeling must consider partitions
- Query patterns are governed by partitioning

Single Row Partitions

When we created the inventory table we did not explicitly defined any partitions.

In this situation:

- Each row is a unique partition
- There is no grouping of data
- Depends on how you expect to retrieve the data

ID	Make	Model	Year
1001	Dodge	Challenger	1971
1002	Ford	Mustang	1968
1003	Chevy	Camaro	1969
1004	Dodge	Daytona	1969
1005	Dodge	Challenger	1972
1006	Ford	Mustang	1971
1007	Dodge	Charger	1969

Grouping Data

We could find some ways to group data in the table.

Groups of data are partitions. In the example cars are grouped by make.

Partitions are stored on the same node in contiguous storage, meaning that it will ve retrieved together at query-time.

ID	Make	Model	Year
1001	Dodge	Challenger	1971
1004	Dodge	Daytona	1969
1005	Dodge	Challenger	1972
1007	Dodge	Charger	1969
1003	Chevy	Camaro	1969
1002	Ford	Mustang	1968
1006	Ford	Mustang	1971

Create Tables with Partitions

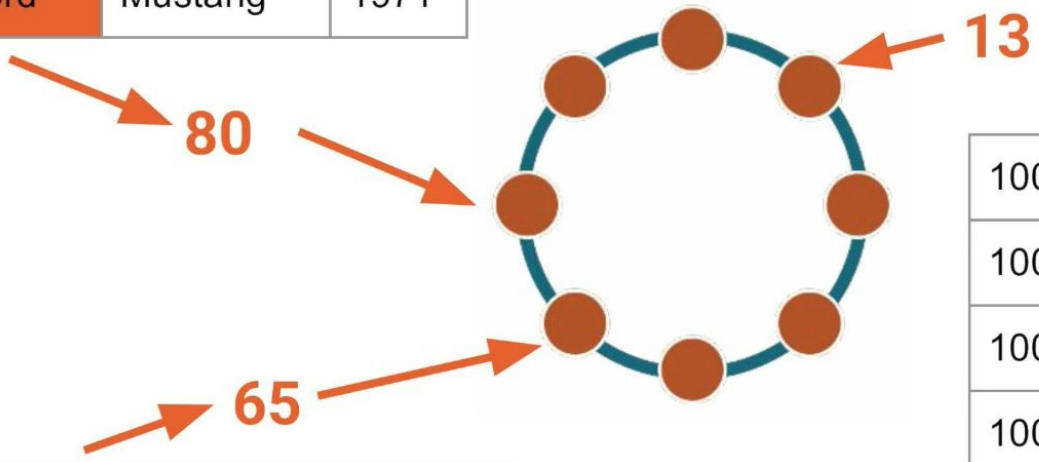
Primary keys are made up of two parts:

1. Column(s) in the inner parentheses of the **PRIMARY KEY** are the *partition key*
2. Remaining columns (if any) are clustering columns

```
CREATE TABLE inventory (
     id int,
     make text,
     model text,
     year int,
     PRIMARY KEY ((make), id)
);
```

Token

1002	Ford	Mustang	1968
1006	Ford	Mustang	1971



1001	Dodge	Challenger	1971
1004	Dodge	Daytona	1969
1005	Dodge	Challenger	1972
1007	Dodge	Charger	1969

1003	Chevy	Camaro	1969

Queries

In Cassandra queries with a WHERE clause must include the partition key.

```
SELECT * FROM inventory WHERE make = ‘Dodge’;
```

ALLOW FILTERING clause enables queries without specifying the partition key, but it is considered an anti-pattern that results froma a data modeling error.

Composite Partition Keys

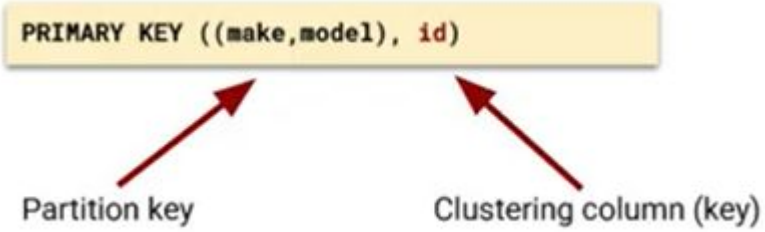
```
CREATE TABLE inventory (
     id int,
     make text,
     model text,
     year int,
     PRIMARY KEY ((make, model), id)
);
```

ID	Make	Model	Year
1001	Dodge	Challenger	1971
1005	Dodge	Challenger	1972
1003	Chevy	Camaro	1969
1007	Dodge	Charger	1969
1002	Ford	Mustang	1968
1006	Ford	Mustang	1971
1004	Dodge	Daytona	1969

Primary Keys and Clustering Columns

Primary Keys uniquely identify rows (in previous example id column was always included in all the PRIMARY KEY clauses), and it is composed of 2 parts:

1. **Partition key** → column(s) used to generate a token and group/distribute rows around the cluster
2. **Clustering columns** → zero or more columns that define order of rows within a partition. Define both the logical and physical storage for data.



Queries with Clustering Columns

Queries must use the partition key and may use clustering columns in WHERE clauses (and when are used they must be used from Left to Right in order).

```
SELECT * FROM inventory WHERE make='Dodge' AND model='Challenger';
SELECT * FROM inventory WHERE make='Dodge' AND model='Challenger' AND id = 1005;
```

Clustering Columns and Order

Default order is ascending, but it can be overridden.

```
CREATE TABLE inventory (
     id int,
     make text,
     model text,
     year int,
     PRIMARY KEY ((make, model), id)
) WITH CLUSTERING ORDER BY (id DESC);
```

Clustering Columns and Order

id: ascending (default) order

```
cqlsh:cars> select * from inventory;

 make  | model       | id   | year
-------+-------------+------+------
 Dodge | Challenger  | 1001 | 1971
 Dodge | Challenger  | 1005 | 1972
  Ford |     Mustang | 1002 | 1968
  Ford |     Mustang | 1006 | 1971
 Dodge |     Daytona | 1004 | 1969
 Dodge |     Charger | 1007 | 1969
 Chevy |      Camaro | 1003 | 1969

(7 rows)
cqlsh:cars>
```

```
PRIMARY KEY ((make,model), id)
```

id: descending order

```
cqlsh:cars> select * from inventory;

 make  | model       | id   | year
-------+-------------+------+------
 Dodge | Challenger  | 1005 | 1972
 Dodge | Challenger  | 1001 | 1971
  Ford |     Mustang | 1002 | 1968
  Ford |     Mustang | 1006 | 1971
 Dodge |     Daytona | 1004 | 1969
 Dodge |     Charger | 1007 | 1969
 Chevy |      Camaro | 1003 | 1969

(7 rows)
cqlsh:cars>
```

```
PRIMARY KEY ((make,model), id DESC)
```

Valid Queries

All Chevy Camaros	`SELECT * FROM cars WHERE make='Chevy' AND model='Camaro';`
All Chevy Camaros highest miles first	`SELECT * FROM cars WHERE make='Chevy' AND model='Camaro' ORDER BY miles DESC;`
All Chevy Camaros with 31000 miles	`SELECT * FROM cars WHERE make='Chevy' AND model='Camaro' AND miles = 31000;`
All Chevy Camaros over 30000 miles	`SELECT * FROM cars WHERE make='Chevy' AND model='Camaro' AND miles > 30000;`

```
PRIMARY KEY ((make, model), miles, year)
```

Invalid Queries

All 1969 Chevy Camaros	`SELECT * FROM cars WHERE make='Chevy' AND model='Camaro' AND year=1969;`	**miles** must be constrained before year
All red Chevy Camaros	`SELECT * FROM cars WHERE make='Chevy' AND model='Camaro' AND color='red';`	**WHERE** clause can only contain key columns
All cars with 31000 miles	`SELECT * FROM cars WHERE miles=31000;`	**WHERE** clause must include partition key
All Camaros	`SELECT * FROM cars WHERE model='Camaro';`	**WHERE** clause must include all columns of partition key

```
PRIMARY KEY ((make, model), miles, year)
```

Queries with ORDER BY

```
PRIMARY KEY ((make, model), miles, year, price)
    WITH CLUSTERING ORDER BY (miles DESC, year ASC, price DESC);
```

Valid ORDER BY clause	Invalid ORDER BY clause
`ORDER BY miles ASC, year DESC;`	`ORDER BY miles ASC, year ASC;`
`ORDER BY miles DESC, year ASC, price DESC;`	`ORDER BY miles ASC, year ASC, price ASC;`
`ORDER BY miles ASC, year DESC, price ASC;`	`ORDER BY miles ASC, year DESC, price DESC;`
`ORDER BY miles DESC;`	`ORDER BY price DESC;`

practice

Practice - 1

Goto: Workshop I

practice

Practice - 2 CRUD

Goto: KillerCoda - CQL

Consistency

Consistency Level

The number of replica nodes that must ack for an operation to succeed.

If there are not enough acks to meet the consistency level, the operation fails.

Consistency level can be set Globally or per individual operations.

Consistency Level	READS	WRITES	How many replicas must acknowledge a request
ANY	N/A	✔	None
ONE	✔	✔	One
TWO	✔	✔	Two
THREE	✔	✔	Three
QUORUM	✔	✔	More than half
ALL	✔	✔	ALL

Set Consistency Level

CONSISTENCY clause is used to set or view consistency level.

```
SELECT * FROM inventory USING CONSISTENCY QUORUM;
```

Reads and Writes

Cassandra will **always** try to write to ALL replicas required to satisfy the replication factor

- The write will succeed if enough replicas ack to satisfy the required consistency level

Cassandra will **only** attempt to read from the number of replicas required to satisfy the consistency level

- If those replicas do not ack, then the read fails

Immediate Consistency

You can achieve immediate consistency in Cassandra with the two following write-read strategies:

- **QUORUM - QUORUM**
- **ALL - ONE**

Hinted Handoff

Hinted Handoff

Coordinator node stores hint to pass data to failed replica nodes on restart.

It only stores hints (for a configurable amount of time, usually 180mins) and if an operation is successful but not every replica responded.

Read Repair

Read Repair (I)

Read repair is an anti-entropy operation that helps to syncronize data in the cluster that is initiated as part of the read operations.

When a read is performed, Cassandra:

1. Coordinator send request to 2 nodes (QUORUM)
2. Nearest node returns data
3. Other node returns checksum
4. If checksum matches data, coordinator returns data

Read Repair (II)

If checksum does not match Coordinator compares timestamps.

If data is newer → coordinator returns data and sends data to the other replica too, to align it.

If checksum is newer → coordinator requests data from replica and returns it, it will also sends it to other replica to align it.

Data Modeling

Introduction

Apache Cassandra data model is based around and **optimized for querying** (query-driven).

It does not support relational data modeling intended for relational databases, for instance join and foreign keys are <u>not part</u> of Cassandra's model.

What is Data Modeling?

Is the process of identifying entities and their relationships.

In **relational databases**, data is placed in normalized tables with foreign keys used to reference related data in other tables. Queries that the application will make are driven by the structure of the tables and related data are queried as table joins.

In **Cassandra**, data is modeled around specific queries and data access patterns which determine the structure and organization of data.

Query-Driven Modeling

- Start from data access patterns, not entity diagrams.
- Each important query should be served by one table.
- All fields needed for a query are grouped in the same table.
- Data for the same entity may appear in multiple tables.

Result: denormalization and data duplication (as well as high write throughput) are expected and intentional. The goal here is to have *very fast reads*.

Primary Key

The choice of the **primary key** and **partition key** is important to **distribute data evenly across the cluster**.

Keeping the number of partitions read for a query to a minimum is crucial, because different partitions could be located on different nodes and the coordinator would need to send a request to each node adding to the request overhead and latency.

An example of partitioning (1)

Consider a variation of table *t* that has two fields constituting the primary key.

```
CREATE TABLE t (
   id int,
   k int,
   v text,
   PRIMARY KEY (id)
);
```

Here the partition key is generated from the primary key *id* for data distribution across the nodes in a cluster.

An example of partitioning (2)

Consider table *t* in which *id* is the only field in the primary key.

```
CREATE TABLE t (
   id int,
   c text,
   k int,
   v text,
   PRIMARY KEY (id,c)
);
```

The first field, *id*, is used to generate the partition key and the second field *c* is the clustering key used for sorting within a partition.

Using clustering keys to sort data makes retrieval of adjacent data more efficient.

An example of partitioning (3)

The partition key could be generated from multiple fields if they are grouped as the first component of a primary key.

```
CREATE TABLE t (
   id1 int,
   id2 int,
   c1 text,
   c2 text
   k int,
   v text,
   PRIMARY KEY
((id1,id2),c1,c2)
);
```

practice

Practice - Examples of Data Modeling

A magazine data set consists of data for magazines with attributes such as magazine id, magazine name, publication frequency, publication date, and publisher.

Query 1 (Q1) - List all the magazine names including their publication frequency

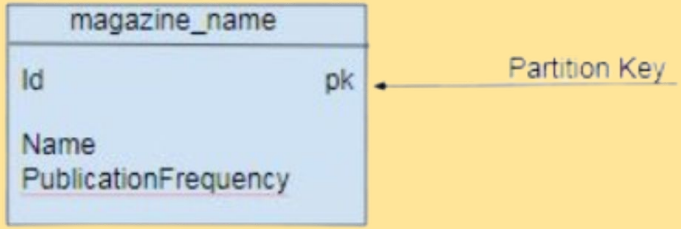
Query 2 (Q2) - List all the magazine names by publisher.

How would you represent a table in Cassandra to respond to each query?

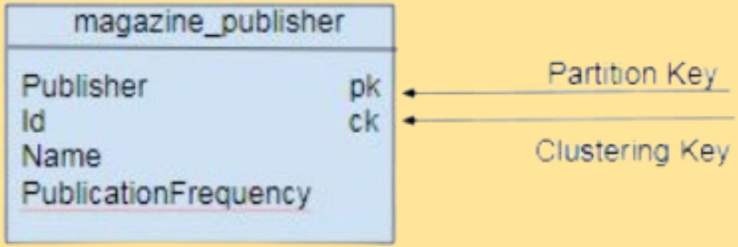
practice

Solutions

Query 1 (Q1)



Query 2 (Q2)



practice

Practice - 3 Data Modeling

Goto: KillerCoda - Data Modeling

Operating Tools

Cassandra Query Language - CQL

SQL-like language, to create, modify, and delete database schema, as well as access data

Other Useful Tools & Commands

Nodetool → used to interact with Cassandra's live control interface, allowing runtime manipulation of many settings from *cassandra.yaml*.

Auditlogviewer → used to view the audit logs.

Fqltool → used to view, replay and compare full query logs.

practice

Extra Cassandra Material

DataStax Academy